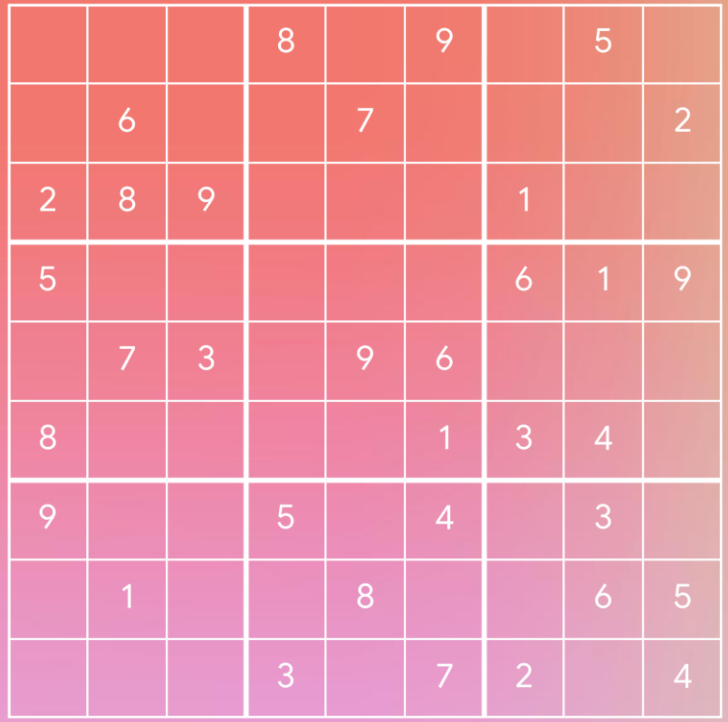
Evolving Sudoku Solvers with OpenEvolve

LLM-Guided Algorithm Discovery



DS5110: Big Data Systems
Isabel Delgado | Razan Habboub | Anna Li

Can an LLM iteratively improve a human-written algorithm?

We examine this research question through applying an **OpenEvolve framework** to Sudoku solving algorithms.

Sudoku: a logic-based, combinatorial number-placement puzzle

Objective: Fill a 9x9 grid with digits so that each column, each row, and each of the nine 3x3 sub-grids contain all the digits from 1 to 9

Baseline Algorithms

Backtracking

- Brute force DFS.
- Scans grid left-to-right and backtracks on conflict.
- Correct but blind to puzzle constraints.

MRV

- Minimum Remaining Values Heuristic.
- Branches on cells with the fewest valid options first.

Naked Singles

- Constraint propagation.
- Fills cells with only one candidate before searching.

https://drive.google.com/file/d/15OngKH2J_kL37xmrxDId5pSap-yoSQve/view?usp=sharing

Evolutionary Framework

Island-Based Evolution

OpenEvolve maintains **diversity** using three separate populations (Islands) with periodic migrations.

MAP-Elites ensures we don't just find the best solution but maintain high-performing programs across different feature dimensions.
Fitness signals are derived from the average backtrack count and solve failures (1.5M penalty).

LLM Evolution Backend

The "Mutator" Model

- Powered by **Gemini 2.5 Flash Lite**. It receives the best parent solver and proposes code refinements.

System Guidance

- Prompts include the **API contract**, scoring formulas, and execution artifacts (stderr/logs) to fix common bugs.

Fitness: Backtrack Efficiency

Evaluation Dataset
150 Puzzles Total → 100 Easy, 25 Hard, 25 Expert
Sourced from the tdoku benchmark suite

All three baselines solve 100% of puzzles given unlimited time.

Under strict 1.5s/puzzle timeout, backtracking solves only **16% of hard, 68% of expert**

Each puzzle contributes its actual backtrack count if solved within 1.5s - otherwise, unsolved or timed-out puzzles contribute a penalty of 1,500,000.

```
avg_bt = sum(puzzle_bt) / 150
combined_score = 1 / (1 + avg_bt / 303708)
```
→ maximized by search

Score approaches 1.0 as backtracks approach zero - perfect propagation, no search.

Results & Observations

3,300x Efficiency Improvement over Baseline

Average per puzzle backtracks dropped from 303k to 91.

80% Solve Rate → 100% Solve Rate

Perfect Solve Rate at iteration 15+, and combined efficiency score increase from 0.5 to 0.9997

Autonomous Logic

Hidden Singles

Identifies digits that can only appear in one cell in a row/column, even with multiple candidates.

Forward Checking

Immediately prunes peer candidate sets after an assignment, failing fast if a cell becomes empty.

Efficient Backtracking

Restore only changed candidates on undo - no full grid copies.

https://drive.google.com/file/d/1Z2D2Yhepr4JDzG7WcxlFehs98FktI-XW/view?usp=sharing

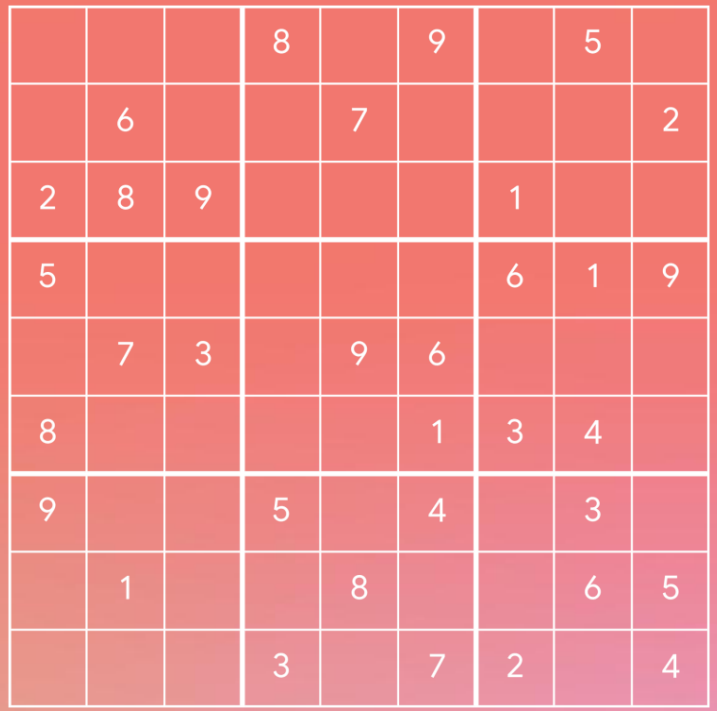
Conclusions

INFRASTRUCTURE & CHALLENGES

- **Backend Migration:** Transitioned from GPT-4o to Gemini mid-project to navigate API quota constraints.
- **Evolutionary Noise:** Managed zero-scoring candidates caused by subtle bugs (e.g., scoping errors) and performance bottlenecks.

RAPID CONVERGENCE

- **Optimal Path:** Peak performance achieved by Iteration 15 and remained unbeaten through 90+ generations.
- **Key Drivers:** Precise fitness signals and well-constrained problem spaces accelerate LLM-guided search.



Thank you!